



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

FPGA Design Using VHDL

Module 6D: VHDL Operators

VHDL Standard Operators

- There exists seven groups of VHDL operators
 1. Binary logical operators (bit and bit array types)
 - `and or nand nor xor xnor`
 2. Relational/Comparison operators (numeric types)
 - `= /= < <= > >=`
 3. Shifts operators (bit array types)
 - `sll, srl, rol, ror`
 4. Adding operators (numeric types)
 - `+` `-` `&` (concatenation)
 5. Unary sign operators (numeric types)
 - `+` `-`
 6. Multiplying operators (numeric types)
 - `*` **`/`** **`mod`** **`rem`**
 7. Miscellaneous operators:
 - `not` `abs` **`**`**

Operators in red and bold are generally non-synthesizable on most FPGA fabrics

Operator Precedence 1

The following is the order of operator precedence

Class						
Highest	1. Miscellaneous op.	**	abs	not		
	2. Multiplying op.	*	/	mod	rem	
	3. Unary operators	+	-			
	4. Addition operators	+	-	&		
	5. Shift operators	sll	srl	rol	ror	
Lowest	6. Relational ops	=	/=	<	<=	> >=
	7. Logical operators	and	or	nand	nor	xor xnor

Operator Precedence 2

Highest	Order of Precedence for Operators					Lowest
Misc	Multiply	Sign	Add	Shift	Relational	Logic

- Evaluation Rules:
 1. Operators in order of highest precedence are evaluated first
 2. Operators of equal precedence are evaluated from left to right, except for the logical and shift operators
 3. Deepest nested parentheses are evaluated first
- Note
 - Use parenthesis liberally for #2
 - Or use variables to break down complex statements

Operator Precedence 3

- Logical operators are the lowest precedence
- Logical operators are evaluated after the rest of the expression
- Logical operators have the same precedence
- Not permitted to mix logical operators
 - As a result, parentheses must be used
- Cannot chain `nand` or `nor`

```
-- ILLEGAL!, can't mix logical
-- operators
result1 <= a and b or c; --!

-- should be...
result2 <= (a and b) or c;
-- or maybe
result3 <= a and (b or c);

-- ILLEGAL!, can't chain nand or nor
-- (nand and nor are not associative)
result4 <= a nand b nand c; --!
result5 <= a nor b nor c;    --!

-- maybe
result6 <= (a nand b) nand c;
-- or
result7 <= a nand (b nand c);

-- will result6 and result7 yield the
-- same results?
```

Operator Precedence 4

--Example 1: ILLEGAL! Why?

if a < b = false then --!

--should be

if (a < b) = false then

--Example 2: this is fine

if a = '1' and b = '0' then

--and is the same as

if (a = '1') and (b = '0') then

--Example 3: ILLEGAL! cannot chain shift operators

result <= a sll 2 srl 2; --!

--should be

result <= (a sll 2) srl 2;

--Example 4: this is okay

if a sll 2 > 5 then

--is equivalent to

if (a sll 2) > 5 then

--Example 5: ILLEGAL, sign can't appear in second term

result <= a + -b; --!

--but this is legal

result <= -b + a;

--Example 6: this...

result <= -a mod b;

--is the same as

result <= -(a mod b);

--Example 7: ILLEGAL, not allowed

result <= a ** b ** c; --!

-- this is fine

result <= a ** (b ** c);

highest

lowest

Class						
1. Miscellaneous op.	**	abs	not			
2. Multiplying op.	*	/	mod	rem		
3. Unary operators	+	-				
4. Addition operators	+	-	&			
5. Shift operators	sll	srl	rol	ror		
6. Relational ops	=	/=	<	<=	>	>=
7. Logical operators	and	or	nand	nor	xor	xnor

highest

lowest

Assume a and b are signed

Assume a, b, and c are natural

Some Examples of Mixed Operators

- What do the following statements evaluate to?

```
signal a, b: unsigned(2 downto 0);  
signal c, d, E1, ..., E4: unsigned(5 downto 0);
```

```
A<="110";  
B<="111";  
C<="011000";  
D<="111011";
```

```
      ②  ①  ④  ③  
E1 <= A & not B or C ror 2;  
      110   000   000110
```

```
E1 <=  
110110
```

```
E2 <= D ror 2 or A & not B;  
E3 <= A * B + C ror 2;  
E4 <= A xnor B & not A;
```

Possibly an exam or
quiz question

Arithmetic Operators: numeric_std

- The `numeric_std` package includes overloaded operators
 - “+” operator is defined to
 - Allow operands of the same type but different lengths
 - Allow addition for different types
 - “>” comparator is defined to allow operands of different types
 - Same applies to other operators mentioned
 - How do you know what operand types are allowed?
 - Look at the operator function for that package (`numeric_std`)
 - https://www.csee.umbc.edu/portal/help/VHDL/packages/numeric_std.vhd

Operator Overloading

- Operator overloading allows different operand types for a given operation (function)
- The compiler resolves which of these function to select based on the types of the inputs
- This selection is hidden from the user For example
 - You **can** add unsigned to natural
 - You **cannot** add unsigned to integer

Example: "+" in numeric_std (unsigned)

```
function "+" (L, R: UNSIGNED) return UNSIGNED is
    constant SIZE: natural := MAX(L'LENGTH, R'LENGTH);
    variable L01 : UNSIGNED(SIZE-1 downto 0);
    variable R01 : UNSIGNED(SIZE-1 downto 0);
begin
    if ((L'LENGTH < 1) or (R'LENGTH < 1)) then
        return NAU;
    end if;
    L01 := TO_01(RESIZE(L, SIZE), 'X');
    if (L01(L01'LEFT)='X') then
        return L01;
    end if;
    R01 := TO_01(RESIZE(R, SIZE), 'X');
    if (R01(R01'LEFT)='X') then
        return R01;
    end if;
    return ADD_UNSIGNED(L01, R01, '0');
end "+";
```

- "+" resizes input operands to same length
- Calls local function ADD_UNSIGNED, shown below, with inputs same length
- ADD_UNSIGNED operates just like you'd expect addition to operate, LSB to MSB, keeping track of carry between bits

```
function ADD_UNSIGNED (L, R: UNSIGNED; C: std_logic) return UNSIGNED is
    constant L_LEFT: integer := L'LENGTH-1;
    alias XL: UNSIGNED(L_LEFT downto 0) is L;
    alias XR: UNSIGNED(L_LEFT downto 0) is R;
    variable RESULT: UNSIGNED(L_LEFT downto 0);
    variable CBIT: std_logic := C;
begin
    for I in 0 to L_LEFT loop
        RESULT(I) := CBIT xor XL(I) xor XR(I);
        CBIT := (CBIT and XL(I)) or (CBIT and XR(I)) or (XL(I) and XR(I));
    end loop;
    return RESULT;
end ADD_UNSIGNED;
```

Example: “+” in numeric_std (signed)

```
function "+" (L, R: SIGNED) return SIGNED is
    constant SIZE: natural := MAX(L'LENGTH, R'LENGTH);
    variable L01 : SIGNED(SIZE-1 downto 0);
    variable R01 : SIGNED(SIZE-1 downto 0);
begin
    if ((L'LENGTH < 1) or (R'LENGTH < 1)) then
        return NAU;
    end if;
    L01 := TO_01(RESIZE(L, SIZE), 'X');
    if (L01(L01'LEFT)='X') then
        return L01;
    end if;
    R01 := TO_01(RESIZE(R, SIZE), 'X');
    if (R01(R01'LEFT)='X') then
        return R01;
    end if;
    return ADD_SIGNED(L01, R01, '0');
end "+";
```

- “+” resizes input operands to same length
- calls local function ADD_UNSIGNED, shown below, with inputs same length
- ADD_UNSIGNED operates just like you’d expect addition to operate, LSB to MSB, keeping track of carry between bits

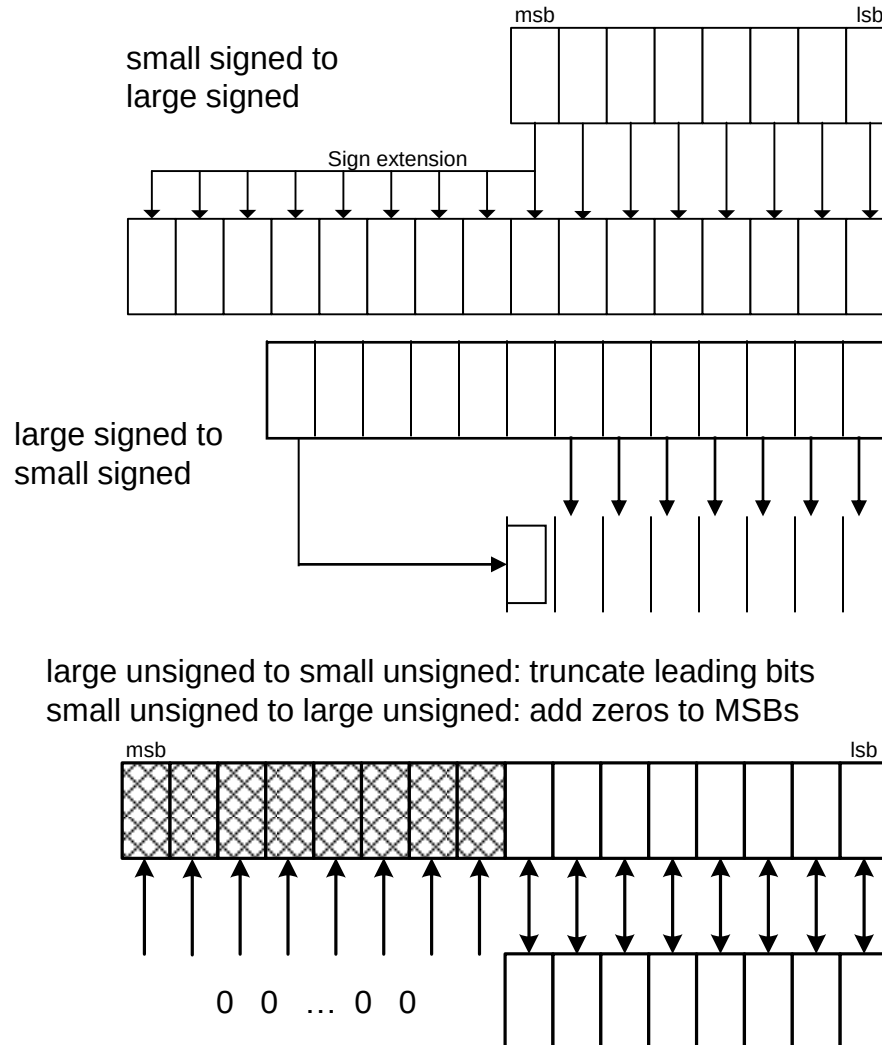
```
function ADD_SIGNED (L, R: SIGNED; C: std_logic) return SIGNED is
    constant L_LEFT: INTEGER := L'LENGTH-1;
    alias XL: SIGNED(L_LEFT downto 0) is L;
    alias XR: SIGNED(L_LEFT downto 0) is R;
    variable RESULT: SIGNED(L_LEFT downto 0);
    variable CBIT: std_logic := C;
begin
    for I in 0 to L_LEFT loop
        RESULT(I) := CBIT xor XL(I) xor XR(I);
        CBIT := (CBIT and XL(I)) or (CBIT and XR(I)) or (XL(I) and XR(I));
    end loop;
    return RESULT;
end ADD_SIGNED;
```

Resize Functions (numeric_std) 1

- function RESIZE (ARG: SIGNED; NEW_SIZE: NATURAL) return SIGNED;
- function RESIZE (ARG: UNSIGNED; NEW_SIZE: NATURAL) return UNSIGNED;

```
function RESIZE (ARG: SIGNED; NEW_SIZE: NATURAL) return SIGNED is
  alias INVEC: SIGNED(ARG'LENGTH-1 downto 0) is ARG;
  variable RESULT: SIGNED(NEW_SIZE-1 downto 0) := (others => '0');
  constant BOUND: INTEGER := MIN(ARG'LENGTH, RESULT'LENGTH)-2;
begin
  if (NEW_SIZE < 1) then
    return NAS;
  end if;
  if (ARG'LENGTH = 0) then
    return RESULT;
  end if;
  RESULT := (others => ARG(ARG'LEFT));
  if BOUND >= 0 then
    RESULT(BOUND downto 0) := INVEC(BOUND downto 0);
  end if;
  return RESULT;
end RESIZE;
```

Resize Functions (numeric_std) 2



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity resizes is
    port (sa: in signed(7 downto 0);
          sb: in signed(15 downto 0);
          sa_large : out signed (15 downto 0);
          sb_small : out signed (7 downto 0);
          ua: in unsigned(7 downto 0);
          ub: in unsigned(15 downto 0);
          ua_large : out unsigned (15 downto 0);
          ub_small : out unsigned (7 downto 0) );
end entity resizes;
```

architecture example of resizes is
begin

--signed length conversions

sa_large <= resize(sa, sa_large'length);

sb_small <= resize(sb, sb_small'length);

--unsigned length conversions

ua_large <= resize(ua, ua_large'length);

ub_small <= resize(ub, ub'length);

end architecture example;

-- conventional resizing could be done using slice
-- ub_small <= ub(ub_small'range);
-- ub_small <= ub(7 downto 0);

Examples of Resize

- Try some examples:

- `a_signed <= "101"; b_signed <= resize(a_signed, 4); --
b_signed = "1101"`
- `b_signed <= "10001"; a_signed <= resize(b_signed, 4);
-- a_signed = "1001";`
- `b_signed <= "11001"; a_signed <= resize(b_signed, 4);
-- a_signed = "1001";`
- Why did we get the same answer for two different operands of `b_signed`?